



UNIVERSIDADE FEDERAL DO CEARÁ
CURSO DE CIÊNCIAS ATUARIAIS
ELEMENTOS DE SÉRIES TEMPORAIS

Prof. Dr. Daniel Tomaz

Modelagem e Previsão do Preço do Ouro

Uma aplicação da metodologia Box–Jenkins à série mensal
do ouro (USD/onça), 2000–2024

Diego Lopes

Fortaleza – CE
2026

1 Introdução

O ouro ocupa posição singular no sistema financeiro internacional, funcionando simultaneamente como reserva de valor, ativo de refúgio em períodos de incerteza e instrumento de proteção (*hedge*) contra a inflação e a desvalorização cambial. Diferentemente de ativos cujo retorno depende de fluxos de caixa futuros, o preço do ouro é governado por fatores macroeconômicos e comportamentais, como taxas de juros reais, política monetária dos principais bancos centrais e aversão global ao risco [1]. Essa natureza confere à sua série de preços uma dinâmica rica, marcada por tendência de longo prazo e por episódios de elevada volatilidade associados a crises.

A relevância de modelar o preço do ouro é dupla. Do ponto de vista da gestão de risco e da atuária, projeções sobre o comportamento do metal subsidiam decisões de alocação de portfólios, precificação de produtos indexados a *commodities* e dimensionamento de reservas [4]. Do ponto de vista metodológico, a série constitui um caso completo para a aplicação das técnicas de análise de séries temporais, por reunir não estacionaridade, ausência de sazonalidade marcante e heterocedasticidade condicional — características recorrentes em séries financeiras [7].

A literatura registra ampla utilização de modelos da família ARIMA na previsão de preços de *commodities* e ativos financeiros, dada a parcimônia e a interpretabilidade desses modelos [2, 5]. A metodologia de Box e Jenkins, em particular, oferece um arcabouço sistemático — identificação, estimação, diagnóstico e previsão — amplamente consolidado na econometria de séries temporais [3, 6].

O presente trabalho tem como objetivo aplicar a metodologia Box–Jenkins à série mensal do preço do ouro, no período de agosto de 2000 a dezembro de 2024, contemplando as etapas de descrição, modelagem, diagnóstico e previsão. A base de dados é composta por 293 observações mensais do preço de fechamento do contrato futuro de ouro (*Gold Futures*, código GC=F), obtidas junto ao Yahoo Finance. Busca-se responder à seguinte questão central: é possível prever o preço mensal do ouro por meio de modelos ARIMA e, em caso afirmativo, qual a confiabilidade dessas previsões fora da amostra? A análise justifica-se tanto pela importância econômica do ativo quanto pela oportunidade de evidenciar, de forma empírica, as possibilidades e os limites dos modelos lineares de séries temporais diante de uma série com forte tendência.

2 Metodologia

2.1 Base de dados

A série analisada corresponde ao preço de fechamento mensal do contrato futuro de ouro (*Gold Futures*, GC=F), expresso em dólares norte-americanos por onça-troy (USD/onça), extraído do Yahoo Finance por meio da biblioteca *yfinance*. O período abrange de agosto de 2000 a dezembro de 2024, totalizando 293 observações mensais.

Em virtude de inconsistências na agregação mensal nativa da fonte — meses ausentes de forma intermitente —, os dados foram coletados em frequência diária e posteriormente agregados ao último preço de fechamento de cada mês, garantindo um índice temporal mensal contínuo e sem lacunas. Para fins de avaliação preditiva, a série foi dividida em conjunto de **treino** (agosto de 2000 a dezembro de 2023, 281 observações) e conjunto de **teste** (janeiro a dezembro de 2024, 12 observações). Todos os testes de raiz unitária e a estimação dos modelos foram conduzidos exclusivamente sobre o conjunto de treino, reservando-se o conjunto de teste para a comparação das previsões com os valores efetivamente observados.

2.2 A metodologia Box–Jenkins

A abordagem de Box e Jenkins [2] estrutura a modelagem de séries temporais por meio de um processo iterativo composto por quatro etapas: identificação, estimação, diagnóstico (verificação) e previsão. O modelo de referência é o ARIMA(p, d, q), que combina componentes autorregressivos (AR) de ordem p , integração (diferenciação) de ordem d e médias móveis (MA) de ordem q . Para uma série $\{y_t\}$, o modelo pode ser escrito, de forma compacta, como

$$\phi(L) (1 - L)^d y_t = \theta(L) \varepsilon_t, \quad (1)$$

em que L é o operador de defasagem, $\phi(L) = 1 - \phi_1 L - \dots - \phi_p L^p$ é o polinômio autorregressivo, $\theta(L) = 1 + \theta_1 L + \dots + \theta_q L^q$ é o polinômio de médias móveis, e ε_t é um ruído branco.

Identificação. A primeira etapa consiste em avaliar a estacionaridade da série, pré-requisito para a modelagem. A estacionaridade é verificada por meio de testes de raiz unitária. Foram empregados os testes *Augmented Dickey–Fuller* (ADF) e *Kwiatkowski–Phillips–Schmidt–Shin* (KPSS) — obrigatórios e com a mesma especificação (constante e tendência) —, complementados pelos testes de Phillips–Perron (PP), DF-GLS (Elliott–Rothenberg–Stock) e Zivot–Andrews (ZA), este último robusto à presença de uma quebra estrutural. Cabe destacar que os testes ADF, PP, DF-GLS e ZA têm como hipótese nula a presença de raiz unitária (não estacionaridade), ao passo que o KPSS inverte essa lógica, testando a estacionaridade como hipótese nula. Caso a série se mostre não estacionária, aplica-se a diferenciação e repetem-se os testes. Em seguida, as funções de autocorrelação (FAC) e de autocorrelação parcial (FACP) da série estacionária orientam a escolha das ordens p e q dos modelos candidatos.

Estimação. Os modelos candidatos têm seus parâmetros estimados por máxima verossimilhança. A comparação entre especificações concorrentes apoia-se nos critérios de informação de Akaike (AIC) e Bayesiano (BIC), que penalizam a complexidade do modelo; valores menores indicam melhor compromisso entre ajuste e parcimônia.

Diagnóstico. Avalia-se, então, a adequação dos resíduos. O teste de Ljung–Box verifica a ausência de autocorrelação residual: resíduos que se comportam como ruído branco indicam que a estrutura de dependência temporal foi adequadamente capturada. Complementarmente, os testes de Jarque–Bera e Shapiro–Wilk avaliam a normalidade dos resíduos, e o teste ARCH-LM investiga a presença de heterocedasticidade condicional.

Previsão. Por fim, os modelos validados geram previsões fora da amostra para o horizonte do conjunto de teste. A acurácia preditiva é mensurada por medidas de erro usuais — erro médio (ME), raiz do erro quadrático médio (RMSE), erro absoluto médio (MAE), erro percentual absoluto médio (MAPE) e erro absoluto escalonado médio (MASE) —, selecionando-se o modelo de melhor desempenho. Em conformidade com as orientações da disciplina, a função `auto_arima` foi utilizada apenas como referência complementar, não substituindo a aplicação criteriosa das etapas descritas.

3 Resultados

3.1 Visualização gráfica e estatísticas descritivas

A Figura 1 apresenta a evolução do preço mensal do ouro no período analisado. Observa-se uma trajetória de alta de longo prazo, interrompida por correções relevantes — como a observada após 2012 —, seguida de forte valorização a partir de 2019, com a série atingindo seu máximo histórico ao final de 2024. Não há evidência de sazonalidade marcante, mas nota-se aumento da dispersão ao longo do tempo, sugerindo não estacionaridade.



Figura 1: Preço mensal do ouro (USD/onça), agosto de 2000 a dezembro de 2024.

Fonte: elaboração própria a partir de dados do Yahoo Finance.

A decomposição da série (Figura 2) confirma a predominância do componente de tendência, ao passo que o componente sazonal apresenta amplitude desprezível frente ao nível da série, e o

componente residual concentra a maior parte da variabilidade de curto prazo.

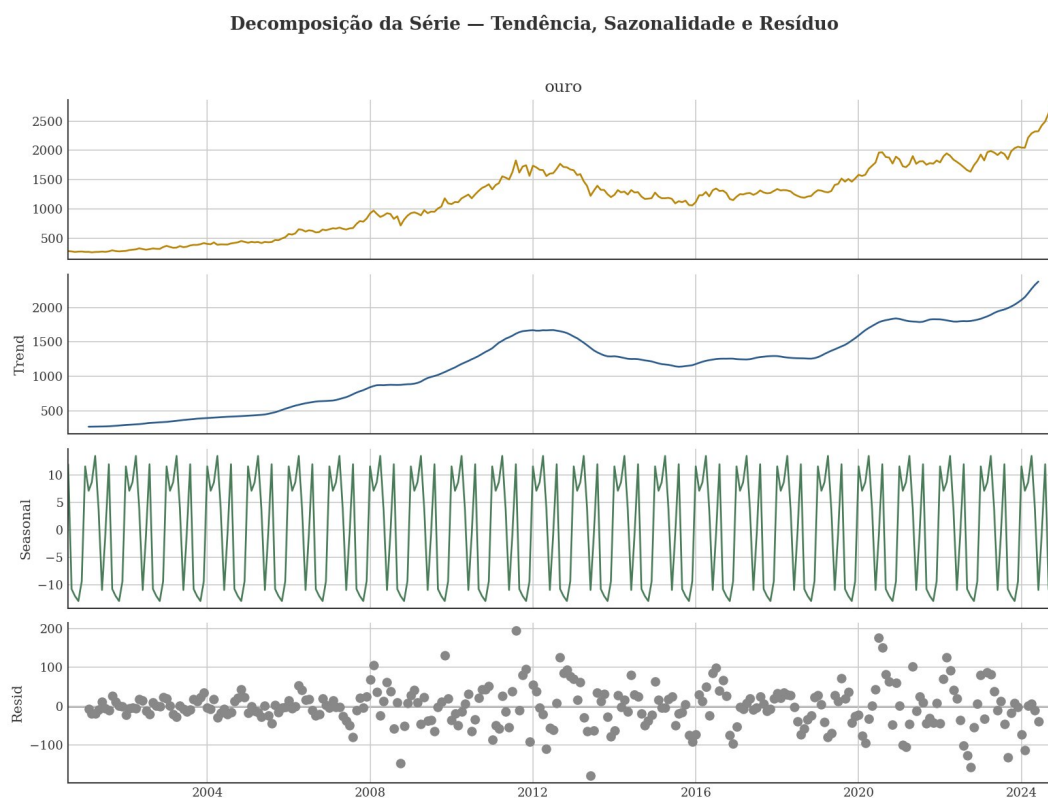


Figura 2: Decomposição aditiva da série: observada, tendência, sazonalidade e resíduo.

Fonte: elaboração própria.

As principais estatísticas descritivas da série completa são apresentadas na Tabela 1. O elevado coeficiente de variação (50,3%) e a ampla distância entre os valores mínimo e máximo reforçam a forte variabilidade do preço do ouro ao longo do período.

Tabela 1: Estatísticas descritivas da série do preço do ouro (USD/onça).

Estatística	Valor
Média	1.158,63
Mediana	1.233,90
Desvio-padrão	583,08
Mínimo	257,90
Máximo	2.738,30
Coef. de variação (%)	50,33

Fonte: elaboração própria.

3.2 Estacionaridade e testes de raiz unitária

A Tabela 2 reúne os resultados dos testes de raiz unitária para a série em nível e em primeira diferença, todos com especificação de constante e tendência. Em nível, o conjunto de testes converge para o diagnóstico de não estacionaridade: ADF, PP, DF-GLS e ZA não rejeitam a hipótese nula de raiz unitária, enquanto o KPSS rejeita a hipótese de estacionaridade. Após a aplicação da primeira diferença, o quadro se inverte por completo, com todos os testes apontando para a estacionaridade da série diferenciada. Conclui-se, portanto, pela ordem de integração $d = 1$, a ser adotada tanto na modelagem quanto na previsão.

Tabela 2: Testes de raiz unitária/estacionaridade (constante e tendência).

Teste	Série em nível		1ª diferença	
	Estatística	Conclusão	Estatística	Conclusão
ADF	-1,86	Não estacionária	-18,92	Estacionária
PP	-2,12	Não estacionária	-18,92	Estacionária
KPSS	0,27	Não estacionária	0,08	Estacionária
ERS – DF-GLS	-1,88	Não estacionária	-18,85	Estacionária
ZA	-4,14	Não estacionária	-19,70	Estacionária

Notas: valores críticos a 5%. Fonte: elaboração própria.

A Figura 3 ilustra a série diferenciada, cujo comportamento — oscilação em torno de uma média aproximadamente constante, sem tendência aparente — é compatível com o diagnóstico de estacionaridade obtido pelos testes.

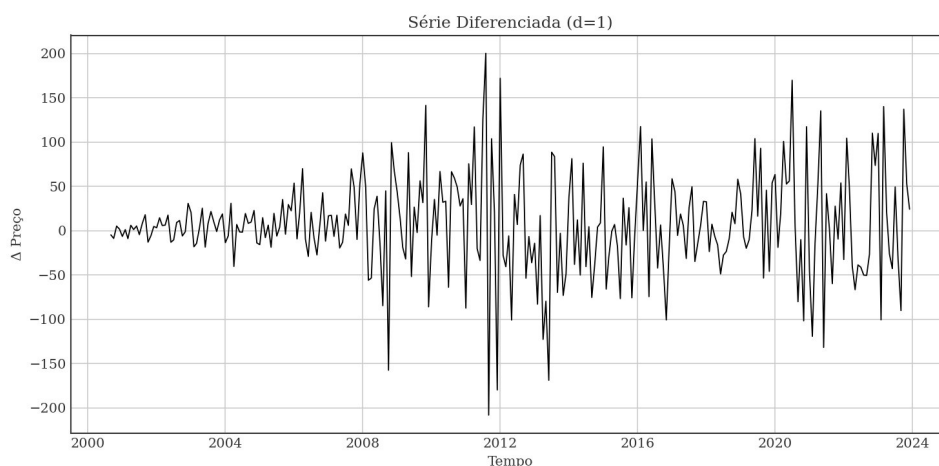


Figura 3: Série em primeira diferença ($d = 1$).

Fonte: elaboração própria.

3.3 Identificação dos modelos

A análise das funções de autocorrelação (FAC) e de autocorrelação parcial (FACP) da série diferenciada (Figura 4) revela truncamento rápido em ambas as funções, com a quase totalidade dos coeficientes contida no interior das bandas de significância. Esse padrão sugere componentes de baixa ordem. Foram, assim, especificados três modelos candidatos circunvizinhos: ARIMA(1,1,0), ARIMA(0,1,1) e ARIMA(1,1,1).

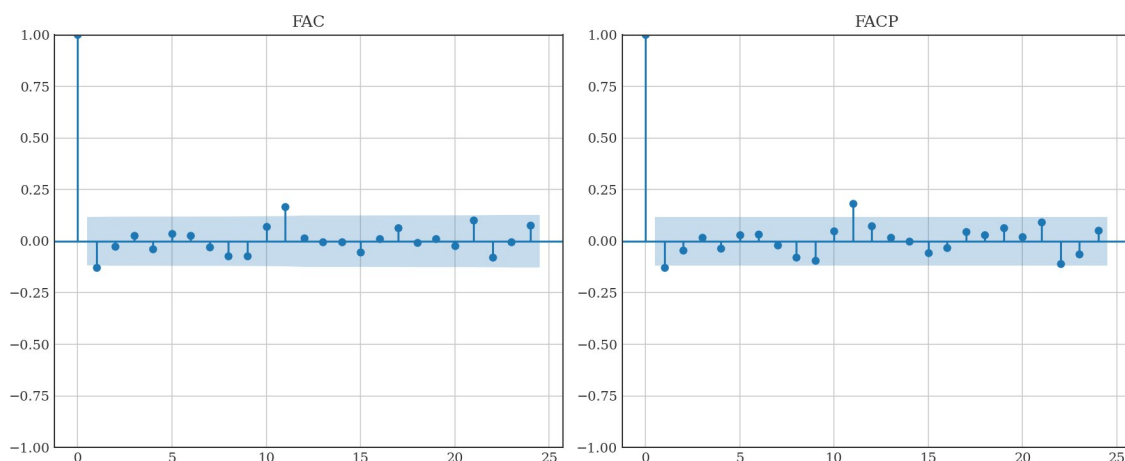


Figura 4: Funções de autocorrelação (FAC) e autocorrelação parcial (FACP) da série diferenciada.

Fonte: elaboração própria.

3.4 Estimação e seleção

A Tabela 3 apresenta os critérios de informação dos modelos estimados. Os três modelos exibem valores de AIC e BIC muito próximos, com leve vantagem do ARIMA(0,1,1), seguido de perto pelo ARIMA(1,1,0). Dada a proximidade e o objetivo preditivo do trabalho, os dois modelos de menor critério foram levados adiante para diagnóstico e previsão.

Tabela 3: Critérios de informação dos modelos estimados.

Modelo	AIC	BIC
ARIMA(1,1,0)	3063,74	3071,01
ARIMA(0,1,1)	3063,62	3070,88
ARIMA(1,1,1)	3065,61	3076,51

Fonte: elaboração própria.

3.5 Diagnóstico dos resíduos

O teste de Ljung–Box (Tabela 4), aplicado com 24 defasagens, não rejeita a hipótese nula de ausência de autocorrelação para nenhum dos modelos ($p > 0,05$). Os resíduos comportam-se, portanto, como ruído branco, indicando captura adequada da dependência temporal da série.

Tabela 4: Teste de Ljung–Box para autocorrelação residual ($gl = 24$).

Modelo	Q*	p-valor	Conclusão
ARIMA(1,1,0)	22,25	0,564	Não rejeita H_0
ARIMA(0,1,1)	21,68	0,599	Não rejeita H_0

Fonte: elaboração própria.

Quanto à distribuição dos resíduos, os testes de Jarque–Bera e Shapiro–Wilk rejeitam a hipótese de normalidade, e o teste ARCH-LM aponta evidência de heterocedasticidade condicional (efeito ARCH). Esses resultados são esperados em séries financeiras e afetam sobretudo a calibração dos intervalos de confiança, não comprometendo a validade da previsão pontual. A modelagem da volatilidade condicional, por meio de especificações da família GARCH, constitui extensão natural para trabalhos futuros.

3.6 Previsão e medidas de erro

A Figura 5 compara as previsões dos dois modelos selecionados com os valores observados ao longo de 2024. Em ambos os casos, a projeção pontual estabiliza-se rapidamente em um patamar próximo a US\$ 2.059, ao passo que o preço efetivo do ouro manteve trajetória de forte alta no período, ultrapassando US\$ 2.700.

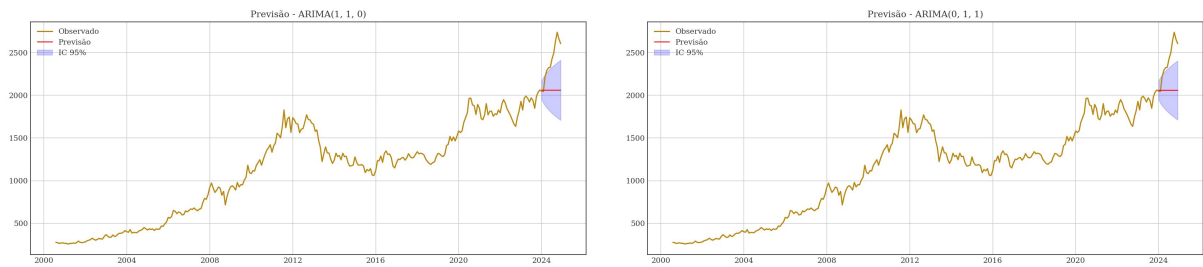


Figura 5: Previsões para 2024: ARIMA(1,1,0) à esquerda e ARIMA(0,1,1) à direita. Linha em ouro: observado; linha em vermelho: previsão; faixa em azul: intervalo de confiança de 95%.

Fonte: elaboração própria.

As medidas de erro no conjunto de teste (Tabela 5) confirmam o descolamento entre previsão e realidade. O modelo ARIMA(1,1,0) apresentou desempenho marginalmente superior em todas as métricas, embora a diferença frente ao ARIMA(0,1,1) seja desprezível. Destaca-se o valor do MASE, muito acima da unidade ($\approx 8,3$), o que indica que, neste horizonte específico, os modelos ARIMA foram superados pela previsão ingênua (*naïve*).

Tabela 5: Medidas de erro dos modelos no conjunto de teste.

Modelo	ME	RMSE	MAE	MAPE (%)	MASE
ARIMA(1,1,0)	341,07	407,17	345,31	13,65	8,32
ARIMA(0,1,1)	342,37	408,27	346,21	13,69	8,34

Fonte: elaboração própria.

Esse resultado merece interpretação cuidadosa. Modelos ARIMA com $d = 1$ projetam, em essência, o último nível observado acrescido de uma deriva constante; em horizontes longos e diante de uma tendência acentuada, como a verificada em 2024, a previsão tende a um patamar quase plano, incapaz de acompanhar a aceleração do preço. O erro elevado reflete, assim, a natureza do período de teste — atipicamente altista — e a limitação intrínseca dos modelos lineares de séries temporais nesse contexto, e não uma falha de especificação. A robustez do diagnóstico de resíduos reforça que, do ponto de vista estatístico, os modelos foram corretamente identificados e estimados.

4 Considerações Finais

A aplicação da metodologia Box–Jenkins à série mensal do preço do ouro permitiu caracterizá-la como um processo integrado de ordem um, $I(1)$: não estacionária em nível e estacionária após a primeira diferença. Os modelos ARIMA(1,1,0) e ARIMA(0,1,1), parcimoniosos e de critérios de informação praticamente equivalentes, capturaram adequadamente a estrutura de dependência temporal, conforme atestado pela ausência de autocorrelação residual.

No exercício de previsão fora da amostra, contudo, os modelos apresentaram erro elevado, em razão da forte valorização do ouro ao longo de 2024 — movimento que projeções lineares com deriva constante não conseguem reproduzir. Esse achado, longe de invalidar o trabalho, evidencia de maneira didática os limites dos modelos ARIMA diante de tendências pronunciadas. Como desdobramentos, sugerem-se a incorporação de modelos da família GARCH para o tratamento da volatilidade condicional e a adoção de horizontes de teste mais curtos ou de esquemas de reestimação em janela móvel (*rolling forecast*), que tendem a melhorar a acurácia preditiva.

Referências

- [1] BAUR, D. G.; LUCEY, B. M. Is gold a hedge or a safe haven? An analysis of stocks, bonds and gold. *The Financial Review*, v. 45, n. 2, p. 217–229, 2010.
- [2] BOX, G. E. P.; JENKINS, G. M.; REINSEL, G. C.; LJUNG, G. M. *Time Series Analysis: Forecasting and Control*. 5. ed. Hoboken: Wiley, 2015.

- [3] BUENO, R. L. S. *Econometria de Séries Temporais*. 2. ed. São Paulo: Cengage Learning, 2018.
- [4] HOOD, M.; MALIK, F. Is gold the best hedge and a safe haven under changing stock market volatility? *Review of Financial Economics*, v. 22, n. 2, p. 47–52, 2013.
- [5] HYNDMAN, R. J.; ATHANASOPOULOS, G. *Forecasting: Principles and Practice*. 2. ed. Melbourne: OTexts, 2018.
- [6] MORETTIN, P. A.; TOLOI, C. M. C. *Análise de Séries Temporais*. 2. ed. São Paulo: Edgard Blücher, 2006.
- [7] TSAY, R. S. *Analysis of Financial Time Series*. 3. ed. Hoboken: Wiley, 2010.

A Código-fonte (Python)

O código a seguir reproduz integralmente a análise, desde a coleta dos dados até a geração das previsões e medidas de erro.

```
1 import yfinance as yf
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from statsmodels.tsa.seasonal import seasonal_decompose
6 from statsmodels.tsa.stattools import adfuller, kpss
7 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
8 from statsmodels.tsa.arima.model import ARIMA
9 from statsmodels.stats.diagnostic import acorr_ljungbox, het_arch
10 from statsmodels.stats.stattools import jarque_bera
11 from scipy.stats import shapiro
12 from arch.unitroot import DFGLS, PhillipsPerron, ZivotAndrews
13 import warnings
14 warnings.filterwarnings("ignore")
15
16 plt.rcParams["figure.figsize"] = (10, 5)
17
18 ticker = "GC=F" # ouro futuro
19 data_diaria = yf.download(ticker, start="2000-01-01", end="2024-12-31",
20                           interval="1d", progress=False)
21
22 serie_diaria = data_diaria["Close"]
23 if isinstance(serie_diaria, pd.DataFrame):
24     serie_diaria = serie_diaria.iloc[:, 0] # forca Series, pega a unica
25     coluna
26 serie_diaria = serie_diaria.dropna()
27 serie_diaria.name = "ouro"
```

```

27
28 # Resample para mensal, usa o ultimo preco de fechamento disponivel em
    cada mes
29 serie = serie_diaria.resample("MS").last()
30 serie.name = "ouro"
31
32 print("Total de observacoes (mensal via resample):", len(serie))
33 print(type(serie))
34
35 full_index = pd.date_range(serie.index.min(), serie.index.max(), freq="
    MS")
36 faltantes_reais = full_index.difference(serie.dropna().index)
37 print(f"Meses ainda ausentes: {len(faltantes_reais)}")
38 if len(faltantes_reais) > 0:
39     print([d.strftime("%Y-%m") for d in faltantes_reais])
40     serie = serie.reindex(full_index).interpolate(method="linear")
41
42 serie = serie.asfreq("MS")
43 serie.name = "ouro"
44
45 print(serie.head())
46 print(serie.tail())
47
48 # Salvar dados brutos
49 serie.to_csv("dados_ouro.csv")
50
51 # Divisao treino/teste
52 ultimo_mes_disponivel = serie.index.max()
53 print(f"\nUltimo mes disponivel na serie: {ultimo_mes_disponivel.
    strftime('%Y-%m')}")
54
55 train = serie[:"2023-12-31"]
56 test = serie["2024-01-01":ultimo_mes_disponivel]
57
58 print("\nTreino:", train.index.min(), "a", train.index.max(), "-", len(
    train), "obs")
59 print("Teste:", test.index.min(), "a", test.index.max(), "-", len(test),
    "obs")
60
61 # INSPECAO VISUAL DA SERIE
62
63 plt.style.use("seaborn-v0_8-whitegrid")
64 plt.rcParams.update({
65     "font.family": "serif",
66     "axes.edgecolor": "#333333",
67     "axes.labelcolor": "#333333",
68     "text.color": "#333333",

```

```

69     "xtick.color": "#333333",
70     "ytick.color": "#333333",
71 })
72
73 fig, ax = plt.subplots(figsize=(11, 5))
74 ax.plot(serie.index, serie.values, color="#B8860B", linewidth=1.6) #
75     tom dourado, tematico
76 ax.fill_between(serie.index, serie.values, serie.values.min(), color="#
77     B8860B", alpha=0.08)
78
79 ax.set_title("Preço do Ouro (USD/onça) Jan/2000 a Dez/2024",
80     fontsize=14, fontweight="bold", pad=15)
81 ax.set_xlabel("Ano", fontsize=11)
82 ax.set_ylabel("Preço (USD)", fontsize=11)
83 ax.spines["top"].set_visible(False)
84 ax.spines["right"].set_visible(False)
85 ax.tick_params(axis="both", labelsize=10)
86
87 plt.tight_layout()
88 plt.savefig("fig1_serie_nivel.png", dpi=150, bbox_inches="tight")
89 plt.show()
90 plt.close()
91
92 # Decomposicao (aditiva, period=12 para dados mensais)
93 decomp = seasonal_decompose(serie, model="additive", period=12)
94 fig = decomp.plot()
95 fig.set_size_inches(11, 8)
96
97 cores = ["#B8860B", "#2E5C8A", "#4A7C59", "#888888"]
98 for axis, cor in zip(fig.axes, cores):
99     for line in axis.get_lines():
100         line.set_color(cor)
101         line.set_linewidth(1.3)
102     axis.spines["top"].set_visible(False)
103     axis.spines["right"].set_visible(False)
104     axis.tick_params(labelsize=9)
105
106 fig.suptitle("Decomposicao da Serie Tendencia, Sazonalidade e Residuo",
107     fontsize=13, fontweight="bold", y=1.02)
108 plt.tight_layout()
109 plt.savefig("fig2_decomposicao.png", dpi=150, bbox_inches="tight")
110 plt.show()
111 plt.close()
112
113 # Estatisticas descritivas
114 desc = pd.DataFrame({
115     "Media": [serie.mean()],

```

```

114     "Mediana": [serie.median()],
115     "Desvio Padrao": [serie.std()],
116     "Minimo": [serie.min()],
117     "Maximo": [serie.max()],
118     "Coef. de Variacao (%)": [100 * serie.std() / serie.mean()],
119 })
120 print("\nEstatisticas descritivas:\n", desc.T)
121 desc.T.to_csv("tabela_estatisticas_descritivas.csv")
122
123 # TESTES DE RAIZ UNITARIA - SERIE EM NIVEL (sobre o treino)
124
125 def adf_test(series, regression="ct"):
126     """ADF com constante e tendencia (ct)"""
127     result = adfuller(series, regression=regression, autolag="AIC")
128     return {"Estatistica": result[0], "Valor Critico (5%)": result[4][
129         "5%"], "p-valor": result[1]}
130
131 def kpss_test(series, regression="ct"):
132     result = kpss(series, regression=regression, nlags="auto")
133     return {"Estatistica": result[0], "Valor Critico (5%)": result[3][
134         "5%"], "p-valor": result[1]}
135
136 def run_root_tests(series, label):
137     print(f"\n=== Testes de raiz unitaria: {label} ===")
138
139     adf_res = adf_test(series)
140     kpss_res = kpss_test(series)
141     dfqls_res = DFGLS(series, trend="ct")
142     pp_res = PhillipsPerron(series, trend="ct")
143     za_res = ZivotAndrews(series, trend="ct")
144
145     tabela = pd.DataFrame({
146         "Teste": ["ADF", "PP", "KPSS", "ERS - DF-GLS", "ZA"],
147         "Estatistica de Teste": [
148             adf_res["Estatistica"],
149             pp_res.stat,
150             kpss_res["Estatistica"],
151             dfqls_res.stat,
152             za_res.stat,
153         ],
154         "Valor Critico (5%)": [
155             adf_res["Valor Critico (5%)"],
156             pp_res.critical_values["5%"],
157             kpss_res["Valor Critico (5%)"],
158             dfqls_res.critical_values["5%"],
159             za_res.critical_values["5%"],
160         ],
161     })

```

```

159         "p-valor": [
160             adf_res["p-valor"],
161             pp_res.pvalue,
162             kpss_res["p-valor"],
163             np.nan, # DF-GLS nao retorna p-valor diretamente
164             za_res.pvalue,
165         ],
166     })
167     print(tabela)
168     return tabela
169
170 tab1 = run_root_tests(train, "Serie em nivel")
171 tab1.to_csv("tabela1_raiz_unitaria_nivel.csv", index=False)
172
173 # Diferenciacao
174 train_diff = train.diff().dropna()
175
176 fig, ax = plt.subplots()
177 ax.plot(train_diff.index, train_diff.values, color="black", linewidth
178         =0.9)
179 ax.set_title("Serie Diferenciada (d=1)")
180 ax.set_xlabel("Tempo")
181 ax.set_ylabel("Delta Preco")
182 plt.tight_layout()
183 plt.savefig("fig3_serie_diferenciada.png", dpi=150)
184 plt.close()
185
186 tab2 = run_root_tests(train_diff, "Serie em primeira diferenca")
187 tab2.to_csv("tabela2_raiz_unitaria_diferenca.csv", index=False)
188
189 # IDENTIFICACAO - FAC E FACP
190
191
192 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
193 plot_acf(train_diff, lags=24, ax=axes[0])
194 axes[0].set_title("FAC")
195 plot_pacf(train_diff, lags=24, ax=axes[1])
196 axes[1].set_title("FACP")
197 plt.tight_layout()
198 plt.savefig("fig4_fac_facp.png", dpi=150)
199 plt.show()
200 plt.close()
201
202
203 # ESTIMACAO DE MODELOS ARIMA CANDIDATOS
204 # Ajuste as ordens (p,d,q) com base na inspecao real da FAC/FACP do seu

```

```

    grafico
205
206 candidatos = [(1, 1, 0), (0, 1, 1), (1, 1, 1)]
207 modelos = {}
208 criterios = []
209
210 for ordem in candidatos:
211     modelo = ARIMA(train, order=ordem).fit()
212     modelos[ordem] = modelo
213     criterios.append({
214         "Modelo": f"ARIMA{ordem}",
215         "AIC": modelo.aic,
216         "BIC": modelo.bic,
217     })
218
219 tab_criterios = pd.DataFrame(criterios)
220 print("\nCritérios de informacao:\n", tab_criterios)
221 tab_criterios.to_csv("tabela3_criterios_informacao.csv", index=False)
222
223 # Escolher os 2 melhores modelos (menor AIC/BIC) para diagnostico e
    previsao
224 melhores = tab_criterios.nsmallest(2, "BIC")["Modelo"].tolist()
225 print("\nModelos selecionados para diagnostico:", melhores)
226
227
228 # DIAGNOSTICO DOS RESIDUOS
229
230 diagnostico_ljungbox = []
231 diagnostico_normalidade = []
232
233 for ordem in candidatos:
234     modelo = modelos[ordem]
235     nome = f"ARIMA{ordem}"
236     residuos = modelo.resid
237
238     # Ljung-Box (lag = 24, comum para dados mensais)
239     lb = acorr_ljungbox(residuos, lags=[24], return_df=True)
240     diagnostico_ljungbox.append({
241         "Modelo": nome,
242         "Q*": lb["lb_stat"].values[0],
243         "gl": 24,
244         "p-valor": lb["lb_pvalue"].values[0],
245         "Conclusao": "Nao rejeita H0" if lb["lb_pvalue"].values[0] >
            0.05 else "Rejeita H0",
246     })
247
248 # Jarque-Bera

```

```

249     jb_stat, jb_p, _, _ = jarque_bera(residuos)
250     # Shapiro-Wilk
251     sw_stat, sw_p = shapiro(residuos)
252     # ARCH-LM
253     arch_stat, arch_p, _, _ = het_arch(residuos)
254
255     diagnostico_normalidade.append({"Teste": "Jarque-Bera", "Modelo":
256                                     nome, "Estatistica": jb_stat, "p-valor": jb_p})
257     diagnostico_normalidade.append({"Teste": "Shapiro-Wilk", "Modelo":
258                                     nome, "Estatistica": sw_stat, "p-valor": sw_p})
259     diagnostico_normalidade.append({"Teste": "ARCH-LM", "Modelo": nome,
260                                     "Estatistica": arch_stat, "p-valor": arch_p})
261
262     tab_ljungbox = pd.DataFrame(diagnostico_ljungbox)
263     tab_normalidade = pd.DataFrame(diagnostico_normalidade)
264     print("\nLjung-Box:\n", tab_ljungbox)
265     print("\nNormalidade/Heterocedasticidade:\n", tab_normalidade)
266     tab_ljungbox.to_csv("tabela4_ljungbox.csv", index=False)
267     tab_normalidade.to_csv("tabela5_normalidade_arch.csv", index=False)
268
269     # PREVISAO E MEDIDAS DE ERRO
270
271     n_test = len(test)
272     previsoes = {}
273     erros = []
274
275     for ordem in candidatos:
276         modelo = modelos[ordem]
277         nome = f"ARIMA{ordem}"
278         forecast = modelo.get_forecast(steps=n_test)
279         pred = forecast.predicted_mean
280         pred.index = test.index
281         previsoes[nome] = pred
282
283         erro = test.values - pred.values
284         me = np.mean(erro)
285         rmse = np.sqrt(np.mean(erro**2))
286         mae = np.mean(np.abs(erro))
287         mpe = np.mean(erro / test.values) * 100
288         mape = np.mean(np.abs(erro / test.values)) * 100
289         # MASE: usa erro absoluto medio do passeio aleatorio (naive) no
290         # treino como benchmark
291         naive_mae = np.mean(np.abs(train.diff().dropna()))
292         mase = mae / naive_mae

```



```

292     erros.append({"Modelo": nome, "ME": me, "RMSE": rmse, "MAE": mae,
293                 "MPE": mpe, "MAPE": mape, "MASE": mase})
294
295 tab_erros = pd.DataFrame(erros)
296 print("\nMedidas de erro:\n", tab_erros)
297 tab_erros.to_csv("tabela7_medidas_erro.csv", index=False)
298
299 # Tabela de observados vs previstos
300 tab_previsto = pd.DataFrame({"Observado": test.values}, index=test.index
301                             )
302 for nome, pred in previsoes.items():
303     tab_previsto[nome] = pred.values
304 print("\nObservado vs Previsto:\n", tab_previsto)
305 tab_previsto.to_csv("tabela6_observado_previsto.csv")
306
307 # Graficos de previsao
308 for ordem in candidatos:
309     nome = f"ARIMA{ordem}"
310     modelo = modelos[ordem]
311     forecast = modelo.get_forecast(steps=n_test)
312     pred_mean = forecast.predicted_mean
313     conf_int = forecast.conf_int()
314
315     #fig, ax = plt.subplots()
316     #ax.plot(serie.index, serie.values, color="black", linewidth=0.8, #
317             #label="Observado")
318
319     fig, ax = plt.subplots(figsize=(11, 5))
320     ax.plot(serie.index, serie.values, color="#B8860B", linewidth=1.6,
321             label= "Observado" ) # tom dourado, tematico
322
323     pred_index = test.index
324     ax.plot(pred_index, pred_mean.values, color="red", linewidth=1.2,
325             label="Previsao")
326     ax.fill_between(pred_index, conf_int.iloc[:, 0], conf_int.iloc[:,
327                     1],
328                     color="blue", alpha=0.2, label="IC 95%")
329     ax.set_title(f"Previsao - {nome}")
330     ax.legend()
331     plt.tight_layout()
332     plt.savefig(f"fig_previsao_{ordem}.png", dpi=150)
333     plt.show()
334     plt.close()
335
336 print("\nMelhor modelo (menor RMSE):", tab_erros.loc[tab_erros["RMSE"].
337             idxmin(), "Modelo"])
338 print("\nScript finalizado. Todos os arquivos (.csv e .png) foram salvos

```

```
no diretorio atual.")
```